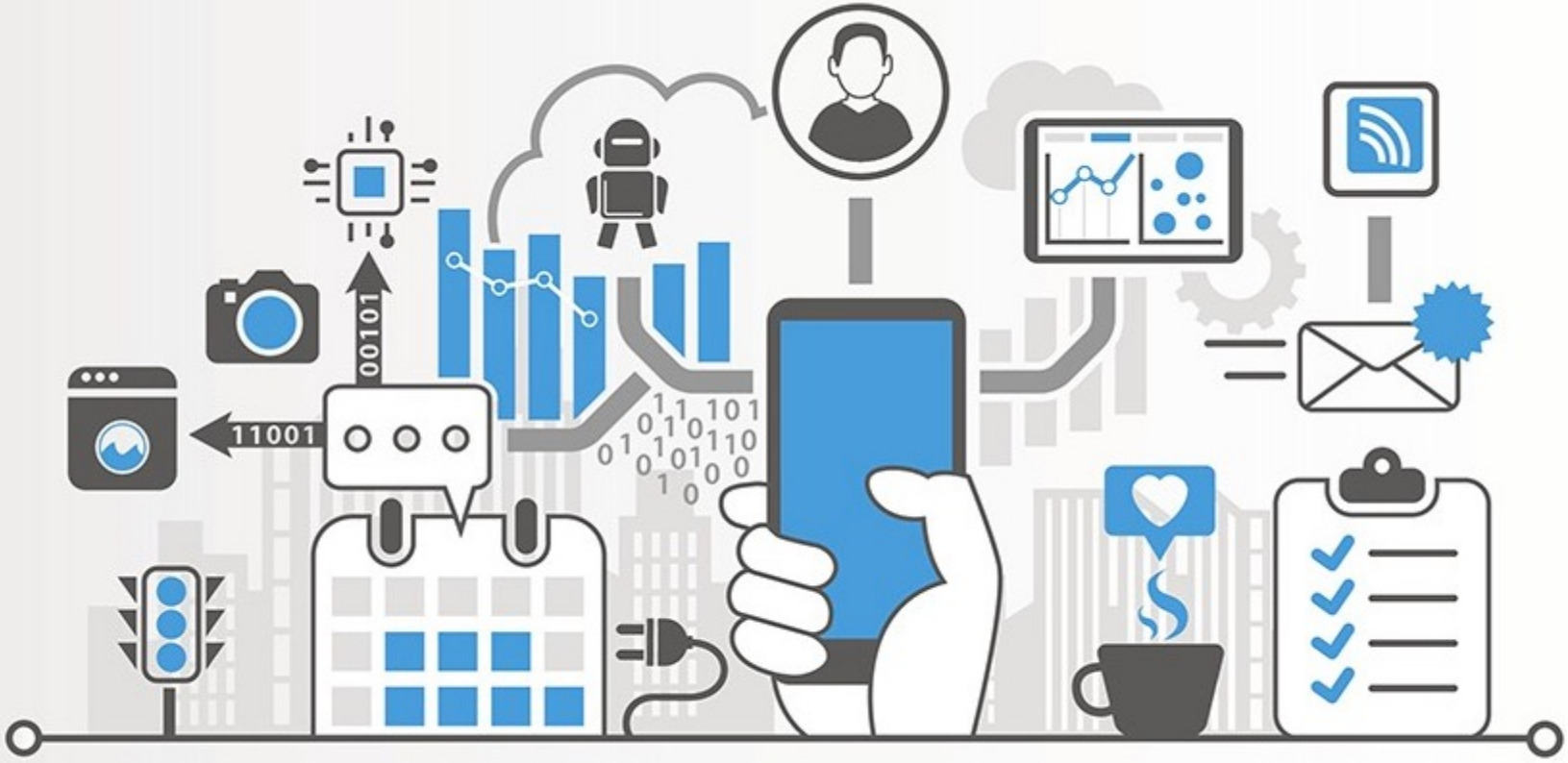
Tecnologie per IoT

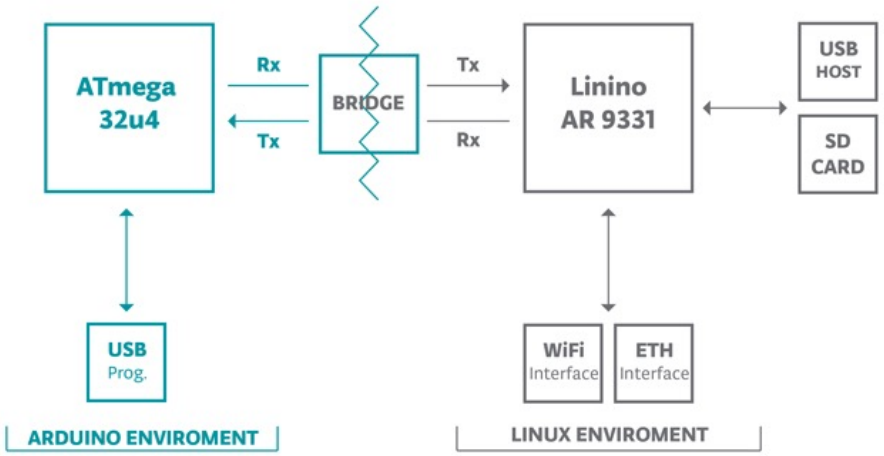
Daniele Pagliari
Luca Barbierato

Lab1: Hardware



Overview

- All three exercises in Part3 use the same HW components
  - One of the two LEDs *(or the internal LED)*
  - The temperature sensor *(or a "virtual sensor")*
  - The Serial interface (for debugging and error reporting)
  - The **WiFi interface** of the Yùn
    - Accessed through the Linino processor
    - Linino and MCU talk through the BRIDGE port
- The goal is to let the Yùn communicate via REST and MQTT



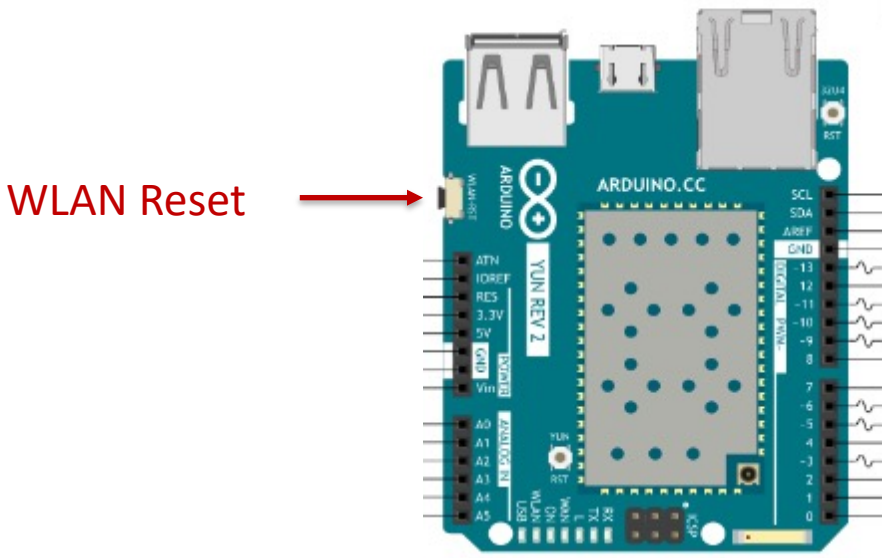
Overview

- Double-check that your Yùn is connected to your smartphone hotspot
  - Must be the same network that your laptop is connected to
  - Recap on how to setup the connection…

ARDUINO YÙN FIRST SETUP

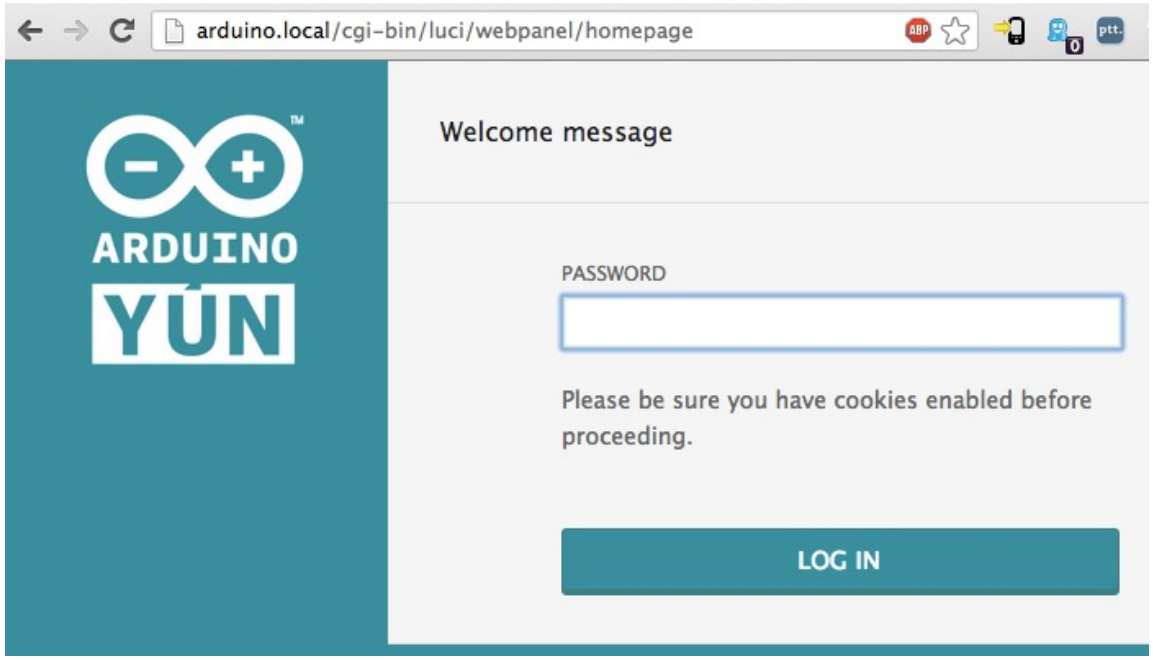
First Setup

- Perform a factory restore holding the **WLAN reset** button for more than 30s



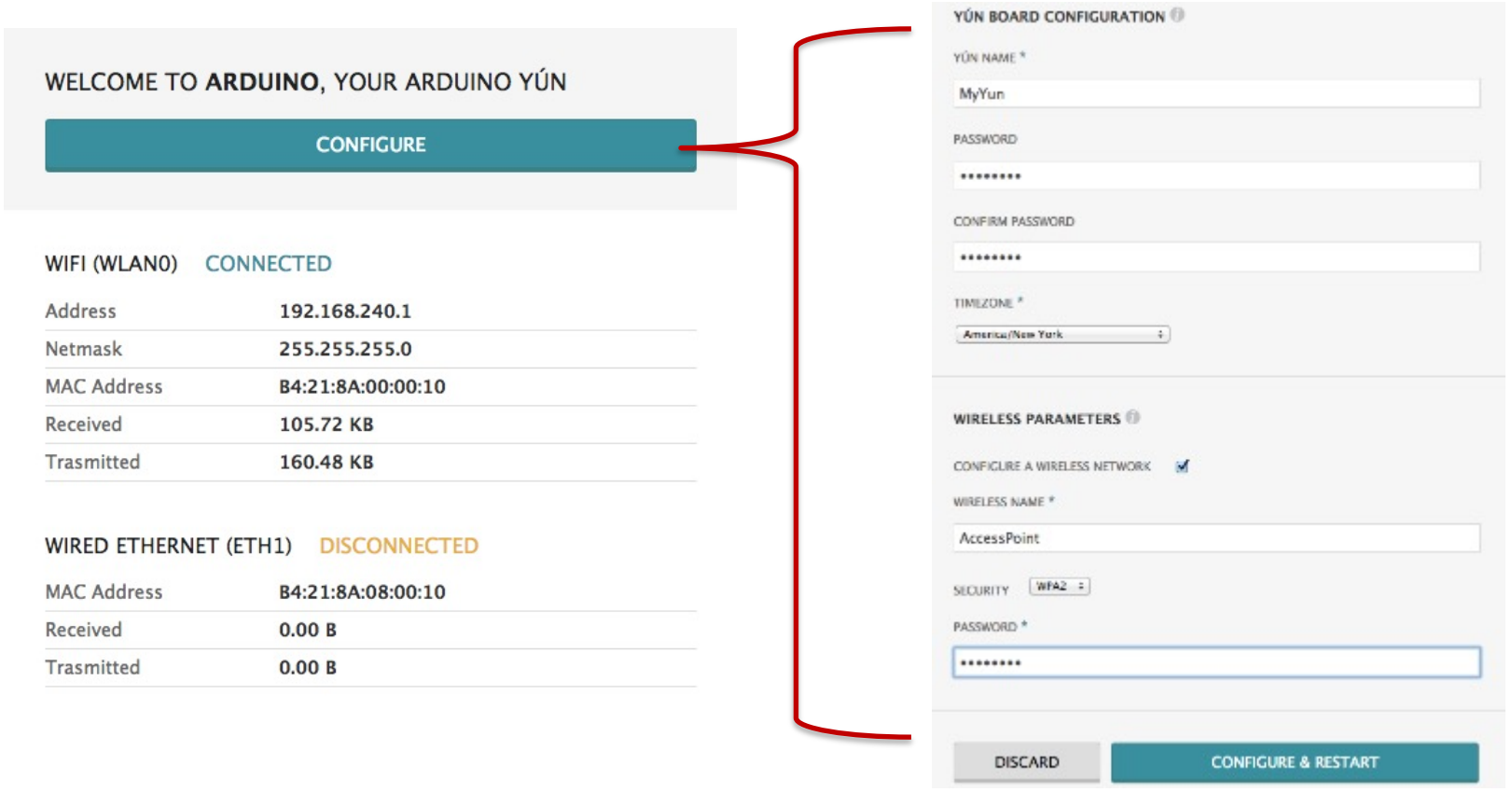
First Setup

- After that the Yùn creates a WiFi network called *ArduinoYun-XXXXXXXXXXXX*
  - Connect to it, open a browser and navigate to `http://arduino.local` or `http://192.168.240.1`
  - You should see this page (default password: **arduino**)



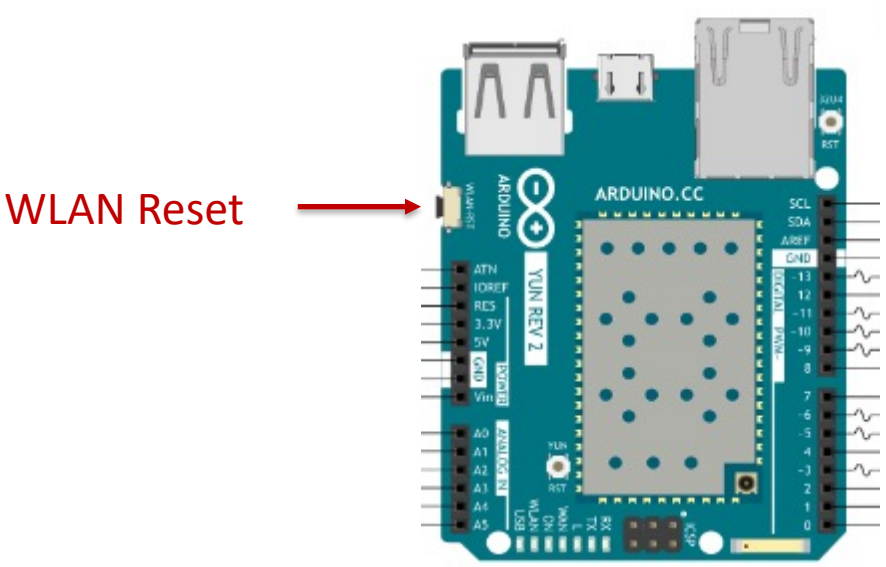
First Setup

- In the following configuration page, follow the instructions to configure the Yùn and let it connect to your home WiFi network or smartphone hotspot.



First Setup

- The Yun will reboot and reconnect to the new network…
- Connect also your laptop to it, and see if you can still access the configuration page at: http://<name>.local
  - Where <name> is the name you gave your board in the previous step



- Other instructions for setup can be found here

PART3: EXERCISE 1

The Bridge Library

- An Arduino library which contains code needed to interface with the Linux processor
  - Many other libraries use “Bridge” for different use cases
- Reference: here
- In this exercise we will use two components of the library
  - BridgeServer (to setup an HTTP server on the Yùn)
  - BridgeClient (to handle individual requests to the server)

New Code Elements

- Include required libraries

```
#include <Bridge.h>
#include <BridgeServer.h>
#include <BridgeClient.h>
```

- Declare global `BridgeServer` instance

```
BridgeServer server;
```

- Setup the server

```
void setup() {
  //...usual setup (pin mode, etc.)...
  pinMode(INT_LED_PIN, OUTPUT);
  digitalWrite(INT_LED_PIN, LOW);
  Bridge.begin();
  digitalWrite(INT_LED_PIN, HIGH);
  server.listenOnLocalhost();
  server.begin();
}
```

Init. bridge conn.

Start server

Use internal LED to understand when the bridge is ready (WiFi connection takes a while on first boot)

New Code Elements

- In `loop()`, process requests using `BridgeClient`

Accept a connection

```
void loop() {
  BridgeClient client = server.accept();

  if (client) {
    process(client);
    client.stop();
  }

  delay(50);
}
```

Close the connection

Avoid overloading with too many connections

New Code Elements

- Read from client as any other `Stream` (exactly as the `Serial` class)

```
void process(BridgeClient client) {

  String command = client.readStringUntil('/');
  command.trim();

  if (command == "led") {
    int val = client.parseInt();
    if (val == 0 || val == 1) {
      digitalWrite(LED_PIN, val);
      printResponse(client, 200, senMlEncode(F("led"), val,F("")));
    } else {

    // etc....
}
```

Parse 1st URL element

Delete \n or similar

Parse 2nd URL element

`F()` around a string constant saves it in FLASH rather than in RAM!!

New Code Elements

- Print the header and body of the HTTP response

```
void printResponse(BridgeClient client, int code, String body) {
  client.println("Status: " + String(code));
  if (code == 200) {
    client.println(F("Content-type: application/json; charset=utf-8"));
    client.println(); //mandatory blank line
    client.println(body); //the response body
  }
}
```

HTTP response code

Here `Content-type` simply helps having a better formatting in the browser (in the next exercise it will be fundamental)

New Code Elements

- The body must be encoded in SenML JSON format

```
{
  “bn”: “Yùn”
  “e”: [
     {
     “n”: <”temperature”>/<”led”>,
     “t”: <timestamp using millis()>,
     “v”: value,
     “u”: “Cel”/null
     }
  ]
}
```

- You can do this “by hand” (it’s just a concatenation of strings):
  - Not very flexible but saves precious memory on the MCUs
  - In the next exercises we’ll see how to do more flexible encoding and (most importantly) decoding with Python on the Linino.

PART3: EXERCISE 2 (PREPARATION)

Connect to the Linino

- For this exercise, you'll need to run commands on Linino (Atheros processor) and transfer files to it from your laptop
- The easiest connection mechanism is through **ssh**:
- Type this command in the macOs/Linux shell or Windows PowerShell:
  - `ssh root@ArduinoName.local`
  - ArduinoName is the name gave to the board during initial setup
- Type the password you set during the Yun Setup:

Connect to the Linino

- You should see this screen:

```
➜  ~ ssh root@arduinodp2.local
root@arduinodp2.local's password:


BusyBox v1.28.3 () built-in shell (ash)

  _______                     ________        __
 |       |.-----.-----.-----.|  |  |  |.----.|  |_
 |   -   ||  _  |  -__|     ||  |  |  ||   _||   _|
 |_______||   __|_____|__|__||________||__|  |____|
          |__| W I R E L E S S   F R E E D O M
 -----------------------------------------------------
 LEDEYun 17.11, r6773+1-8dd3a6e
 -----------------------------------------------------
root@ArduinoDP2:~#
```

Copy files to the Linino

- Copy the content of the "`linino_root.zip`" folder onto the Linino's `/root` folder using `scp`
  - ssh's remote file copy utility

- Command example (run on your laptop):

  scp –r ./linino_root/* root@ArduinoName.local:/root

Copy files to the Linino

- New “status” of the Linino /root folder:

```
root@ArduinoDP2:~# ls
configure.sh  paho          urllib3
root@ArduinoDP2:~#
```

- Next, run the `./configure.sh` script.

Configure the Linino

- This will install Python3 and some utility libraries using the Linino package manager (opkg).

```
root@ArduinoDP2:~# ./configure.sh
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/targets/ar71xx/generic/packages/Packages.gz
Updated list of available packages in /var/opkg-lists/ledeyun_core
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/targets/ar71xx/generic/packages/Packages.sig
Signature check passed.
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/base/Packages.gz
Updated list of available packages in /var/opkg-lists/ledeyun_base
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/base/Packages.sig
Signature check passed.
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/arduino/Packages.gz
Updated list of available packages in /var/opkg-lists/ledeyun_arduino
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/arduino/Packages.sig
Signature check passed.
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/custom/Packages.gz
Updated list of available packages in /var/opkg-lists/ledeyun_custom
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/custom/Packages.sig
Signature check passed.
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/juci/Packages.gz
Updated list of available packages in /var/opkg-lists/ledeyun_juci
Downloading http://downloads.arduino.cc/openwrtyun/17.11/packages/packages/mips_24kc/juci/Packages.sig
Signature check passed.
```

Next Steps

- Now you can write and run Python3 programs on the processor (with some limitations on the usable libraries).

- Check the available disk space on the processor:

```
root@ArduinoDP2:~# df -h
Filesystem                Size      Used Available Use% Mounted on
/dev/root                 8.8M      8.8M         0 100% /rom
tmpfs                    29.3M    688.0K     28.6M   2% /tmp
/dev/mtdblock5            5.6M      4.8M    756.0K  87% /overlay
overlayfs:/overlay        5.6M      4.8M    756.0K  87% /
tmpfs                   512.0K         0    512.0K   0% /dev
root@ArduinoDP2:~#
```

This is the available Flash space (unless you have an external SD)

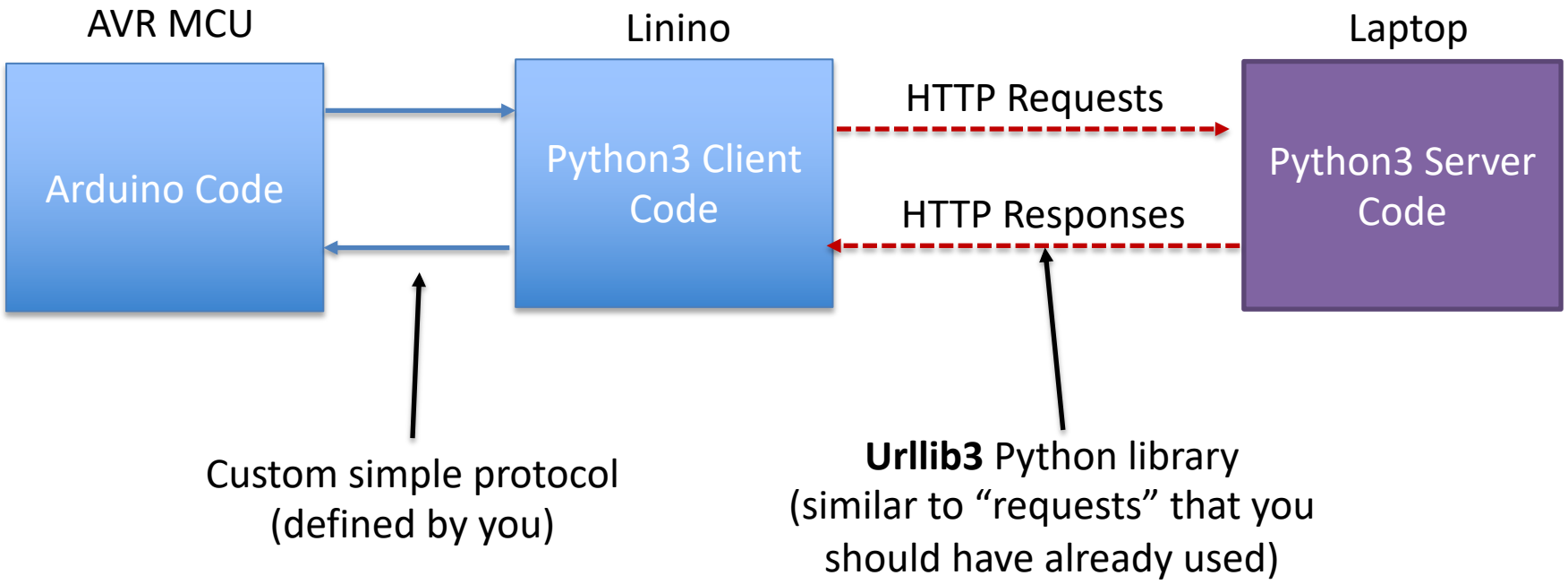
Testing Process

- Suggested step-by-step testing process:

1. **Write the Python3 script on your laptop**
2. **Test in on your laptop**
3. **Test it on Linino as a "standalone" program:**
   - Transfer it to the board with scp
   - Connect with ssh and run it directly from the shell
4. **Write the Arduino "master" program** and launch the Linino Python script through the "Process.h" library.
   - Remember that the AVR MCU is the "master" of the sensors.

PART3: EXERCISE 2

The Organization



Why all this complication?

- Handling HTTP (or MQTT) communication from the AVR in a more “transparent” way would be possible, but the AVR memory is very limited…
  - Handling encoding/decoding of multiple JSON formats and multiple requests types easily leads to memory overflows…
  - Still uses the Atheros under the hood.

- So, we’ll code the “internet-related” part explicitly in Python on the Atheros

The Python3 Client

```python
import json
import urllib3

if __name__ == '__main__':
    http = urllib3.PoolManager()
    while(True):
        msg = input()
        msg = msg.split(':')
        if msg[0] == 'T':
            try:
                val = float(msg[1].strip())
            except:
                print("Cannot convert to float")
                continue
            data = {'temperature': val}
            json_data = json.dumps(data).encode('utf-8')
            r = http.request('POST', 'http://192.168.92.141:8080/log',
                    body=json_data, headers={'Content-Type': 'acpplication/json'})
            print(str(r.status))
        else:
            print("Unrecognized message:", msg)
```

Read a simple message from stdin, in the form:
"<type-of-message>:<value>"
For example, a temperature value can be sent as:
"T:20.5"

The Python3 Client

```python
import json
import urllib3

if __name__ == '__main__':
    http = urllib3.PoolManager()
    while(True):
        msg = input()
        msg = msg.split(':')
        if msg[0] == 'T':
            try:
                val = float(msg[1].strip())
            except:
                print("Cannot convert to float")
                continue
            data = {'temperature': val}
            json_data = json.dumps(data).encode('utf-8')
            r = http.request('POST', 'http://192.168.92.141:8080/log',
                    body=json_data, headers={'Content-Type': 'acpplication/json'})
            print(str(r.status))
        else:
            print("Unrecognized message:", msg)
```

Error check & sanitize
(can be refined)

The Python3 Client

```python
import json
import urllib3

if __name__ == '__main__':
    http = urllib3.PoolManager()
    while(True):
        msg = input()
        msg = msg.split(':')
        if msg[0] == 'T':
            try:
                val = float(msg[1].strip())
            except:
                print("Cannot convert to float")
                continue
            data = {'temperature': val}
            json_data = json.dumps(data).encode('utf-8')
            r = http.request('POST', 'http://192.168.92.141:8080/log',
                    body=json_data, headers={'Content-Type': 'acpplication/json'})
            print(str(r.status))
        else:
            print("Unrecognized message:", msg)
```

Transform to JSON string (UTF-8 encoded)
Mine is simplified, yours should be in SenML format.

The Python3 Client

```python
import json
import urllib3

if __name__ == '__main__':
    http = urllib3.PoolManager()
    while(True):
        msg = input()
        msg = msg.split(':')
        if msg[0] == 'T':
            try:
                val = float(msg[1].strip())
            except:
                print("Cannot convert to float")
                continue
            data = {'temperature': val}
            json_data = json.dumps(data).encode('utf-8')
            r = http.request('POST', 'http://192.168.92.141:8080/log',
                    body=json_data, headers={'Content-Type': 'acpplication/json'})
            print(str(r.status))
        else:
            print("Unrecognized message:", msg)
```

Send the request with urrlib3

The Python3 Client

```python
import json
import urllib3

if __name__ == '__main__':
    http = urllib3.PoolManager()
    while(True):
        msg = input()
        msg = msg.split(':')
        if msg[0] == 'T':
            try:
                val = float(msg[1].strip())
            except:
                print("Cannot convert to float")
                continue
            data = {'temperature': val}
            json_data = json.dumps(data).encode('utf-8')
            r = http.request('POST', 'http://192.168.92.141:8080/log',
                    body=json_data, headers={'Content-Type': 'acpplication/json'})
            print(str(r.status))
        else:
            print("Unrecognized message:", msg)
```

Server IP may change, of course…check your laptop's

The Python3 Client

```python
import json
import urllib3

if __name__ == '__main__':
    http = urllib3.PoolManager()
    while(True):
        msg = input()
        msg = msg.split(':')
        if msg[0] == 'T':
            try:
                val = float(msg[1].strip())
            except:
                print("Cannot convert to float")
                continue
            data = {'temperature': val}
            json_data = json.dumps(data).encode('utf-8')
            r = http.request('POST', 'http://192.168.92.141:8080/log',
                    body=json_data, headers={'Content-Type': 'acpplication/json'})
            print(str(r.status))
        else:
            print("Unrecognized message:", msg)
```

Print response code on stdout

Server

- You also have to modify the cherrypy servers developed in Exercises 1 and 2 of the SW lab to handle both POST and GET requests to the resource:

  `http://<PC IP Address>:<port>/log`

- Nothing special there, you just need to store all logged data and do some JSON `loads()` and `dumps()`….

Test Client on Laptop

Client

```
→  lab_3.2 python3 lab_3.2.py
T:20
200
T:21
200
C:4
Unrecognized message: ['C', '4']
T:T
Cannot convert to float
T:19
200
```

Server

```
→  lab_3.2 python3 server.py
[21/May/2022:23:21:08] ENGINE Bus STARTING
[21/May/2022:23:21:08] ENGINE Started monitor thread 'Autoreloader'.
[21/May/2022:23:21:08] ENGINE Serving on http://0.0.0.0:8080
[21/May/2022:23:21:08] ENGINE Bus STARTED
192.168.92.141 - - [21/May/2022:23:28:43] "POST /log HTTP/1.1" 200 - "" "python-urllib3/1.26.9"
192.168.92.141 - - [21/May/2022:23:28:51] "POST /log HTTP/1.1" 200 - "" "python-urllib3/1.26.9"
192.168.92.141 - - [21/May/2022:23:29:12] "POST /log HTTP/1.1" 200 - "" "python-urllib3/1.26.9"
```

Test on Linino (from shell)

Client

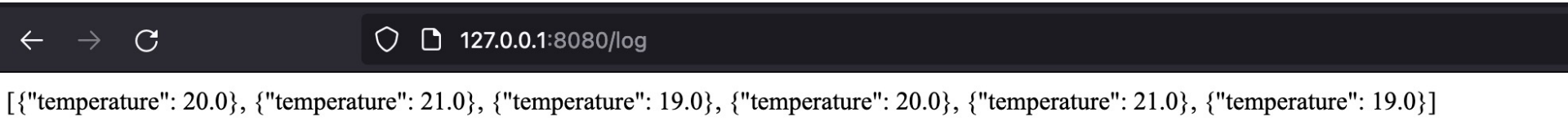
```
root@ArduinoDP2:~# python3 lab_3.2.py
T:20
200
T:21
200
C:4
Unrecognized message: ['C', '4']
T:T
Cannot convert to float
T:19
200
```

Server

```
192.168.92.122 - - [21/May/2022:23:39:01] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [21/May/2022:23:39:09] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [21/May/2022:23:39:23] "POST /log HTTP/1.1" 200 - "" ""
```

Get request from browser

- Server shows list of logged values:



Arduino Code

- What is missing?
  - You want the HTTP POST requests to be sent by the Arduino **upon sensor reading**…
  - Not by you manually through ssh.

- The **Process.h** library comes in handy:
  - It allows to invoke Linino programs from the AVR
- Include it in you Arduino script.

```
#include <Bridge.h>
#include <Process.h>
```

- And create a global Process instance:

```
Process p;
```

Arduino Code

- In the setup function:

```
void setup() {
  Serial.begin(9600);
  while(!Serial);
  Serial.println("Serial initialized!");

  pinMode(INT_LED_PIN, OUTPUT);
  digitalWrite(INT_LED_PIN, LOW);
  Bridge.begin();
  digitalWrite(INT_LED_PIN, HIGH);
  Serial.println("Bridge initialized!");

  p.begin("python3");
  p.addParameter("/root/lab_3.2.py");
  p.runAsynchronously();
}
```

Initialize the Bridge…
(needed by Process)

Run the python script asynchronously
(don't wait for termination)

Arduino Code

- In the loop function:

```
void loop() {
  temp = readTempSensor();
  String msg = "T:" + String(temp);
  p.println(msg);

  Serial.print("Sent command: ");
  Serial.println(msg);

  while(p.available() > 0) {
    char c = p.read();
    Serial.print(c);
  }

  delay(2000);

}
```

Format the message

Write to the process stdin

(it's a serial port under the hood)

Read process stdout

Arduino Code

Client (Serial Monitor)

```
Serial initialized!
Bridge initialized!
Sent command: T:20.50
Sent command: T:20.00
Sent command: T:21.00
Sent command: T:20.50
Sent command: T:20.50
200
200
200
200
Sent command: T:20.00
200
Sent command: T:21.00
200
Sent command: T:20.50
200
Sent command: T:20.50
200
Sent command: T:20.00
200
```

There may be some initial “buffering”

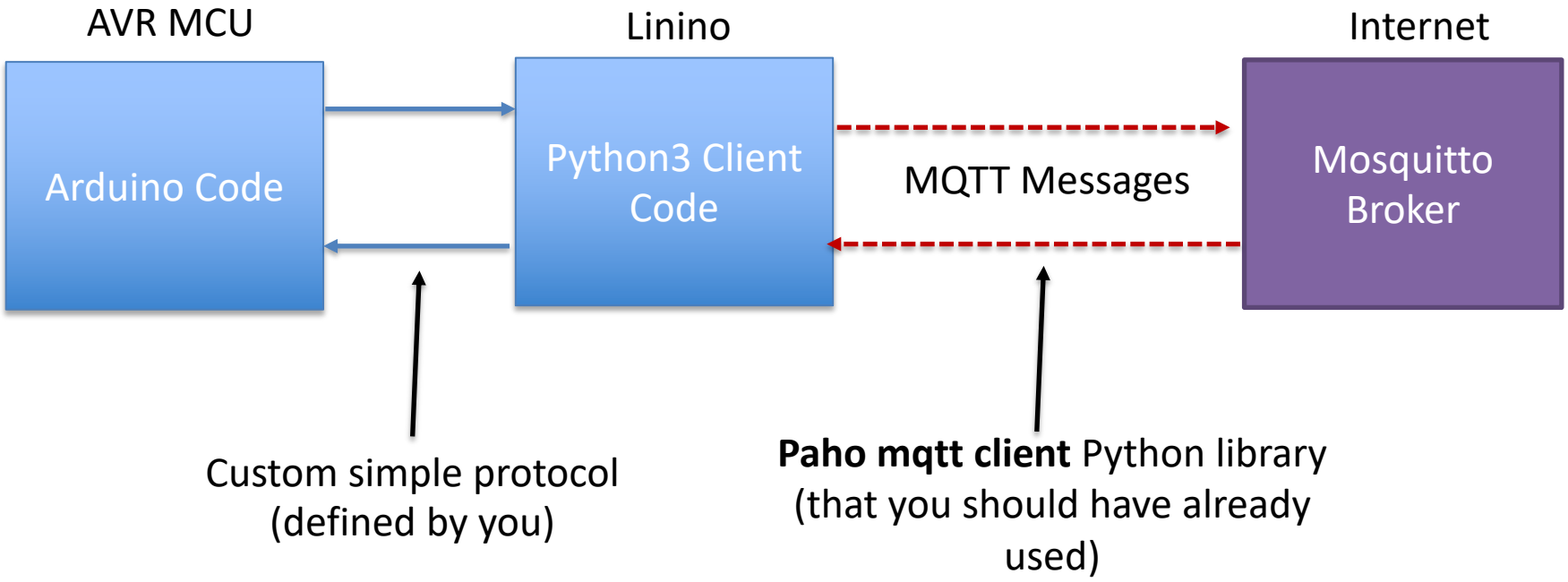
Server

```
192.168.92.122 - - [22/May/2022:00:01:49] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:01:51] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:01:53] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:01:55] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:01:57] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:00] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:02] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:04] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:06] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:08] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:10] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:12] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:14] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:16] "POST /log HTTP/1.1" 200 - "" ""
192.168.92.122 - - [22/May/2022:00:02:18] "POST /log HTTP/1.1" 200 - "" ""
```

[{"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 19.0}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 19.0}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0},
{"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0},
{"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0},
{"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0},
{"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0},
{"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0},
{"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0},
{"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5},
{"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5},
{"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0},
{"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5},
{"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0},
{"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5},
{"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0},
{"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}, {"temperature": 21.0}, {"temperature": 20.5}, {"temperature": 20.5}, {"temperature": 20.0}]

PART3: EXERCISE 3

Structure Similar to 3.2



The Python3 Script

- Import required libraries and pre-format dictionaries for SenML encoding.

```
import paho.mqtt.client as mqtt
import json
import time

# Dictionary for Measurement JSON
MESSAGE = {
    "bn": "YunGroupX",
    "e": []
}
MEASURE = {
    "n": "temperature",
    "u": "Cel",
    "t": 0,
    "v": 0.0
}
```

The Python3 Script

- Handler of incoming messages on subscribed topics

```
def on_message(client, userdata, message):
    topic = message.topic.split('/')
    msg = message.payload.decode('utf-8')
    if topic[-1] == 'command':
        msg = json.loads(msg)
        if "e" not in msg:
            # etc: other error checking here
            return
        peripheral = msg["e"][0]["n"]
        value = msg["e"][0]["v"]
        if peripheral != 'led':
            return
        print("L:"+str(value))
```

Commands for the MCU are formatted in the usual way and sent on stdout

The Python3 Script

- Handler of incoming messages on subscribed topics

```
if __name__ == '__main__':
    client = mqtt.Client()
    client.on_message = on_message
    client.connect('test.mosquitto.org', 1883)
    client.subscribe('/tiot/group0/command')
    client.loop_start()
    while(True):
        msg = input()
        msg = msg.split(':')
        if msg[0] == 'T':
            try:
                val = float(msg[1].strip())
            except:
                print("E:1")
                continue
            MEASURE["t"] = time.time()
            MEASURE["v"] = val
            MESSAGE["e"] = [MEASURE]
            json_data = json.dumps(MESSAGE).encode('utf-8')
            client.publish('/tiot/group0', json_data)
        else:
            print("E:2")
```

Errors must be encoded as well now….

Test as before

- First on laptop, then on Linino as “standalone”.
- Finally, connect with Arduino code.

Testing Exercise 3.3

- To test the exercise, you'll need to install the Mosquitto client (or another MQTT client) on your laptop (instructions can be found here)

- Subscribe command (to read temperature logs):

```
mosquitto_sub -h test.mosquitto.org -t '/tiot/group0/'
```

- Publish command (to turn ON the LED):

```
mosquitto_pub -h test.mosquitto.org -t '/tiot/group0/command' -m
'{"bn": "Yun", "e": [{"n": "led", "t": null, "v": 1, "u":
null}]}'
```

Arduino Code

- Same setup() as before, just change the command name:

```
p.begin("python3");
p.addParameter("/root/lab_3.3.py");
p.runAsynchronously();
```

- Same loop() as before, just change how you handle incoming data from the command stdout:

```
if(p.available() > 0){
  processCommand();
}
```

Parse LED messages and handle error messages

Final Result

```
Sent command: T:20.50
Sent command: T:20.00
Sent command: T:21.00
Sent command: T:20.50
Received command:L:1

Sent command: T:20.50
Sent command: T:20.00
Received command:L:0

Sent command: T:21.00
Sent command: T:20.50
Sent command: T:20.50
```

```
→  ~ mosquitto_sub -h test.mosquitto.org -t '/tiot/group0'
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172428.2178771, "v": 20.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172430.2810755, "v": 21.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172432.3439422, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172434.4077556, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172436.4712045, "v": 20.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172438.5339644, "v": 21.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172440.5969949, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172442.6599414, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172444.7239335, "v": 20.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172446.7869639, "v": 21.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172448.8497865, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172450.9324257, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172452.996938, "v": 20.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172455.0617971, "v": 21.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172457.1230781, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172460.1960535, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172462.2616744, "v": 20.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172465.331336, "v": 21.0}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172467.438318, "v": 20.5}]}
{"bn": "YunGroupX", "e": [{"n": "temperature", "u": "Cel", "t": 1653172469.508672, "v": 20.5}]}
```

```
→  ~ mosquitto_pub -h test.mosquitto.org -t '/tiot/group0/command' -m '{"bn": "YunTest", "e": [{"n": "led", "t": null, "v": 1, "u": null}]}'
→  ~ mosquitto_pub -h test.mosquitto.org -t '/tiot/group0/command' -m '{"bn": "YunTest", "e": [{"n": "led", "t": null, "v": 0, "u": null}]}'
→  ~
```